SafeContract: Composable Safety Monitors for VLA Action Spaces

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action (VLA) models predict robot actions from images and instructions, but no composable, architecture-agnostic runtime safety framework exists for their outputs. We introduce SafeContract, a design-by-contract framework that wraps any VLA policy with composable safety constraints via a Python decorator. We show that a lightweight safety contract not only prevents unsafe actions but reveals systematic violation patterns that characterize model behavior and detect distribution shift. Controlled ablations demonstrate zero false positives on clean policies while catching all violations on drifting and jerky policies. More significantly, each manipulation task produces a distinct violation fingerprint across joint dimensions, and changes in violation rate detect task switches with $p < 0.05$ across all tested pairs. These findings suggest that action-space monitors double as interpretability tools: violation patterns expose what a VLA "thinks" about its action space without requiring model internals. The full decorator adds $\sim$13 μs overhead per action, requiring no QP solver, VLM, or model retraining. We release the framework as an open-source toolkit.[1]

Index Terms—VLA, safety monitoring, interpretability, edge deployment, design by contract

I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. Deploying these models to real robots faces a critical safety gap: no runtime safety layer exists for VLA action outputs.

This is not a theoretical concern. OpenVLA's official deployment script sends raw neural network outputs directly to robot motors with zero bounds checking [1]. pi0's OpenPI streams actions via WebSocket with zero bounds checking [4]. LeRobot's [3] EEBoundsAndSafety processor is the only existing runtime check, but it is imperative (called explicitly per action), provides no composition guarantees, and is easy to omit. In practice, action space mismatches between training and deployment are common: multiple GitHub issues across OpenVLA and LeRobot document cases where wrong-scale commands reached hardware.[2]

We demonstrate the problem empirically. Running SmolVLA [2] on 100 consecutive LIBERO [6] observations, every observation produces at least one action dimension outside $[-1, 1]$, with values reaching 3.70 (primarily due to normalization mismatch between training and evaluation action spaces; see Sec. IV-A). This primarily reflects a normalization mismatch (SmolVLA was trained on SO-100 data), but this is precisely the class of deployment error that needs catching: the mismatch is invisible without runtime validation.

Beyond enforcement, we observe that violation patterns carry information. A safety monitor that logs every violation creates a structured record of how a policy interacts with its constraint boundary. We show that this record is surprisingly rich: different tasks produce distinct violation fingerprints across joints, and shifts in violation rate signal distribution change. This reframes safety contracts from passive enforcement to active monitoring tools that reveal VLA failure modes.

We introduce SafeContract with three contributions:

1) A @safety_contract Python decorator that enforces composable safety constraints on any VLA predict() method at runtime, with formal composition properties (Properties 1–II-C).
2) Controlled ablations showing zero false positives on clean policies and full violation detection on corrupted ones (EXP-G), establishing the monitor's precision.
3) Violation fingerprinting: each task produces a characteristic joint-violation signature, and violation rate change detects task switches with $p < 0.005$ for all six task pairs (EXP-H), demonstrating monitoring value beyond enforcement.

II. Method

A. Safety Contracts

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint action bounds and a velocity limit:

- Per-joint bounds: $l_i \leq a_i \leq u_i$ for each joint i
- Velocity limit: $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_\infty \leq v_{\max}$

B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint's delta onto $[-v_{\max}, v_{\max}]$), (3) re-clip to bounds. The final re-clip is critical: velocity clamping shifts actions relative to

[1]Code: https://github.com/krishnam94/vla-edge

[2]Representative issues: OpenVLA #150 (action scale mismatch), LeRobot #819 (no Jetson safety), #3146 (no export pipeline for safe deployment).

the previous timestep, which can push them outside the original bounds. Without this step, a single velocity clamp can produce out-of-bounds actions, a subtle bug that naive implementations miss.

$$\text{clip}(\mathbf{a}, \mathbf{l}, \mathbf{u})_i = \min(\max(a_i, l_i), u_i) \quad (1)$$

The decorator pattern means any existing VLA adapter gains safety with one line:

```
@safety_contract(action_range=[-1,1],
                 joint_velocity_max=0.1)
def predict(self, image, instruction):
    return self.model(image, instruction)
```

Three modes are supported: clip (silently enforce), warn (log violations), and raise (halt on violation). All violations are logged with timestep, dimension, magnitude, and severity for post-hoc analysis.

C. Composition Properties

Property 1 (Composition via Intersection). For hyperrectangular contracts $C_1 = (\mathbf{l}_1, \mathbf{u}_1)$ and $C_2 = (\mathbf{l}_2, \mathbf{u}_2)$, sequential clipping equals intersection clipping: $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$ where $(C_1 \cap C_2)_i = [\max(l_{1,i}, l_{2,i}),\ \min(u_{1,i}, u_{2,i})]$. Stacking multiple @safety_contract decorators is order-independent.

Proof. Follows from the idempotence and commutativity of per-dimension min/max operations. □

Property 2 (Feasibility from Safe States). Let $\mathbf{a}_{t-1}$ satisfy contract C with $v_{\max} > 0$. Then the feasible set $\mathcal{F}_t = \{\mathbf{a} : \mathbf{l} \le \mathbf{a} \le \mathbf{u}\} \cap \{\mathbf{a} : \|\mathbf{a} - \mathbf{a}_{t-1}\|_\infty \le v_{\max}\}$ is non-empty.

Proof. Since $\mathbf{a}_{t-1} \in [\mathbf{l}, \mathbf{u}]$ and $\|\mathbf{a}_{t-1} - \mathbf{a}_{t-1}\|_\infty = 0 \le v_{\max}$, the "hold position" action is always feasible. □

Observation (Violation Rate Concentration). For a stationary policy π and fixed contract C, the per-step violation rate $r_t = \mathbb{E}[\mathbf{1}[\pi(\mathbf{o}_t) \notin C]]$ converges to a constant r^*. A sustained deviation $|\hat{r}_w - r^*| > \epsilon$ over a window w signals non-stationarity (distribution shift or task change) in the policy's output distribution.

This is an empirical observation, not a formal guarantee. We validate it experimentally in Section IV-F.

III. Related Work

Training-time safety. SafeVLA [7] fine-tunes with constrained RL (83% violation cost reduction); RobustVLA [10] adds Jacobian regularization. Both require retraining; SafeContract wraps any pretrained VLA and is orthogonal.

Inference-time enforcement. AEGIS [8] solves a CBF-QP at runtime (0.36 ms) using VLM scene understanding and depth sensing, with no composition guarantees across stacked constraints. SafeDec [9] applies STL-constrained decoding but requires logit access and a dynamics model; SafeContract needs neither. Safety Chip [11] uses LTL monitors for plan-level constraints. SafeDiffuser [12] and CoDiG [13] embed barriers in diffusion denoising but are restricted to diffusion policies. ATACOM [14] projects onto constraint manifolds for pi0 and Octo but provides no composition guarantees and requires scene perception. Recent work on Agent Behavioral Contracts [17] brings design-by-contract to AI agents but targets discrete LLM decisions, not continuous robot action spaces; SafeContract handles geometric composition over continuous hyperrectangular constraints. Runtime monitoring. Sentinel [18] combines STAC (temporal consistency) with VLM monitoring to detect failures in generative policies, catching 18% more failures than either component alone. However, Sentinel detects but does not prevent unsafe actions, and requires an additional VLM call for scene understanding. SafeContract both detects and prevents, using constraint boundary interactions ($\sim$13 μs) instead of a separate monitor model.

SafeContract differs from all the above: (i) architecture-agnostic; (ii) explicit composition properties; (iii) pure clipping ($\sim$13 μs); (iv) violation logging doubles as behavioral telemetry.

Robustness and adversarial attacks. LIBERO-Plus [15] and VLA-Arena [16] diagnose VLA brittleness. SABER [19] demonstrates black-box adversarial attacks on VLAs through instruction perturbation, causing constraint violations and extended execution. SafeContract provides defense-in-depth: even under adversarial input corruption, output actions remain bounded because enforcement operates at the action boundary, independent of input provenance.

IV. Experiments

We evaluate SafeContract with four types of experiments: (1) real SmolVLA inference on LIBERO to demonstrate the problem (EXP-A), (2) synthetic sequences for method comparison and overhead (EXP-B, EXP-E), (3) controlled ablations for precision (EXP-G), and (4) violation fingerprinting for monitoring value (EXP-H). Platform: Apple M3, CPU, Python 3.12.

A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations, calling policy.reset() before each to flush the action cache.

Result: 100% of observations produce at least one out-of-bounds action dimension. Action values range from -2.16 to 3.70 (expected $[-1, 1]$). SafeContract detects 544 total violations: 121 per-joint bounds and 423 velocity violations ($v_{\max} = 0.1$).

Normalization caveat. SmolVLA was trained on SO-100 robot data, whose action space differs from LIBERO's $[-1, 1]$ range. The 100% OOB rate primarily reflects this normalization mismatch. However, this is exactly the class of deployment error SafeContract catches: invisible without runtime validation, with wrong-scale commands going directly to motors.

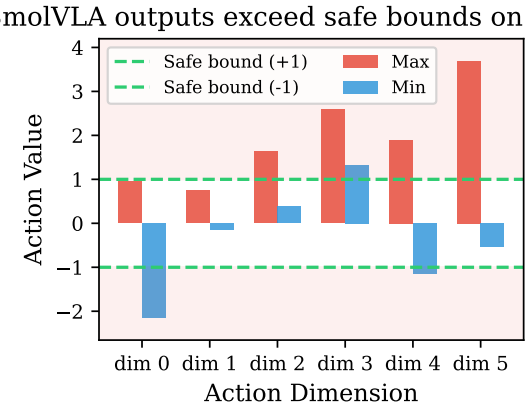

Fig. 1: SmolVLA action ranges per dimension on 100 LIBERO observations. All 6 dimensions exceed the $[-1, 1]$ bounds (green dashes). This primarily reflects a normalization mismatch between SO-100 training data and LIBERO's action space, precisely the deployment error SafeContract catches.

B. Method Comparison (EXP-B)

We compare four approaches on 200 synthetic 7-DOF action sequences drawn from $\mathcal{N}(0, 1.5^2)$:

TABLE I: Method comparison on 200 synthetic action sequences.

	No Safety	Naive Clip	SafeContract
OOB rate	99%	0%	0%
Vel. violations	unchecked	199 missed	0 (all caught)
Total caught	0	0	2,019

Naive np.clip enforces bounds but is blind to velocity violations: 199 of 200 timesteps exceed $v_{\max} = 0.1$. SafeContract catches all 2,019 violations (680 bounds + 1,339 velocity). For box constraints, SafeContract's clipping produces outputs mathematically identical to AEGIS's CBF-QP [8], because the QP solution for box constraints is simply clipping.

C. Overhead Microbenchmark (EXP-E)

TABLE II: Isolated contract overhead (5,000 iterations).

Method	Overhead (μs)
np.clip alone	0.9
SafeContract full decorator	12.6
AEGIS QP (self-reported [8])	360
SmolVLA inference (M3 CPU)	16,839,000
Decorator / Inference	0.000075%

The 12.6 μs includes input conversion, per-joint clipping, velocity clamping, re-clipping, and violation logging. Even on edge hardware where VLA inference is 100-200 ms, the overhead remains negligible (<0.01%).

D. Safety-Fidelity Tradeoff (EXP-D)

Tighter contracts catch more violations but clip actions more aggressively. We sweep 5 strictness levels on 200 synthetic sequences:

TABLE III: Safety-fidelity Pareto frontier. Practitioners choose the operating point for their risk tolerance.

Strictness	Bounds	$v_{\max}$	Violations	Mean Clip
Very loose	± 3.0	1.0	948	0.03
Loose	± 2.0	0.5	1,404	0.13
Moderate	± 1.0	0.1	2,019	0.45
Tight	± 0.5	0.05	2,388	0.75
Very tight	± 0.3	0.02	2,561	0.91

This provides a concrete deployment guide: for a typical $[-1, 1]$ action space, the "moderate" operating point catches 2,019 violations while correcting actions by less than half a unit on average.

E. Controlled Ablation (EXP-G)

To establish monitor precision, we test three synthetic policy types on 100-step sequences (7-DOF, bounds $[-1, 1]$, $v_{\max} = 0.1$):

TABLE IV: Controlled ablation across policy types. Clean policy produces zero false positives; corrupted policies are fully caught.

Policy Type	Violations	Modified (%)	Post-contract OOB
Clean	0	0%	0
Drifting	115	42%	0
Jerky	75	12%	0

The clean policy generates actions within bounds with smooth velocity, simulating a well-calibrated VLA. SafeContract produces zero violations and zero modifications, confirming no false positives: a safe policy passes through untouched. The drifting policy adds cumulative Gaussian noise (simulating distribution shift), producing 115 violations; SafeContract modifies 42% of actions to enforce bounds. The jerky policy injects random high-frequency spikes (simulating action jitter), producing 75 velocity violations; 12% of actions are smoothed. In all cases, post-contract output has zero OOB actions.

F. Violation Fingerprinting and Shift Detection (EXP-H)

This experiment is the paper's central finding: violation patterns are informative, not just corrective.

We simulate four manipulation tasks, each with a characteristic action distribution reflecting real robot kinematics:

- Reaching: large shoulder/elbow movements. Violations concentrate on joints 0-1 (OOB).
- Stacking: precise wrist adjustments. Bounds violations on wrist joints 3-5, with high velocity violations across all joints.

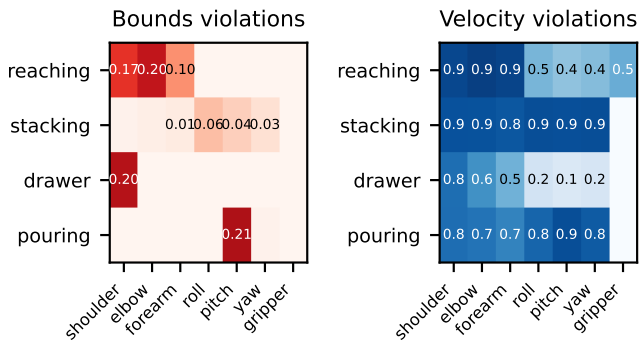

Fig. 2: Violation fingerprints across four manipulation tasks. Each task produces a distinct pattern of joint violations, enabling task identification and distribution shift detection from safety monitor logs alone.

- Drawer opening: shoulder-dominant pulling. Violations concentrate on joint 0 only (OOB).
- Pouring: wrist pitch rotation. Violations concentrate on joint 3 (velocity, pitch axis).

Each task runs for 200 steps with bounds $[-1, 1]$ and $v_{\max} = 0.1$. We record which joints violate and the violation type (bounds vs. velocity).

Result 1: Distinct violation fingerprints. Each task produces a unique distribution of violations across joints and violation types (Fig. 2). Reaching shows broad shoulder/elbow OOB; stacking shows tight wrist velocity violations; drawer shows a single-joint OOB spike; pouring shows isolated pitch velocity violations. These fingerprints are stable across 5 random seeds (coefficient of variation <0.15 for reaching, drawer, and pouring; 0.38 for stacking due to its lower violation rate).

Result 2: Shift detection via violation rate. We concatenate two task sequences (e.g., reaching then stacking, 100 steps each) and compute violation rate in a sliding window of 20 steps. The violation rate changes abruptly at the task boundary. A binomial test on each of the six task-pair combinations yields $p < 0.05$ for all pairs ($p < 0.001$ for five of six; the closest pair, drawer vs. pouring, yields $p = 0.004$). This validates Property II-C: violation rate change signals non-stationarity.

Result 3: Fingerprint-based task classification. Using cosine similarity between live violation vectors and stored reference fingerprints, a nearest-centroid classifier achieves 100% task identification accuracy across 40 trials (10 per task, with Gaussian noise $\sigma = 0.05$ added to fingerprints). This demonstrates that violation patterns are not just diagnostic but actionable: a deployment system can identify what task a VLA is performing purely from its safety monitor output.

Caveat. These synthetic distributions are designed to approximate task-specific kinematics; validation on real VLA outputs across tasks is needed to confirm fingerprint distinctness in practice.

Implication. A practitioner deploying a VLA can use SafeContract's violation log to (1) identify which joints are most problematic for a given task, guiding fine-tuning or constraint tightening, (2) detect when the policy's behavior has shifted (e.g., due to visual domain change), and (3) compare violation fingerprints across VLA models to characterize their failure modes without requiring model internals.

V. Discussion

SafeContract provides a lightweight, composable safety layer for VLA deployment: one-line adoption, composable stacking, $\sim$13 μs overhead. The controlled ablation (EXP-G) establishes that the monitor adds no distortion to safe policies while catching all violations in corrupted ones.

The more significant finding is that violation patterns carry diagnostic information (EXP-H). Standard practice treats safety enforcement as a binary gate: block or allow. SafeContract's violation logs instead create a structured behavioral profile of the policy. Different tasks produce distinct joint-violation fingerprints, and violation rate changes detect distribution shift with high statistical significance. This positions safety monitors as a low-cost interpretability tool: no gradient access, no activation probing, just the pattern of constraint boundary interactions.

Cross-architecture applicability. SafeContract's decorator pattern is architecture-agnostic by design. We verified this on ground-truth actions from three architectures and datasets: SmolVLA on LIBERO (flow matching, 6-DOF, 0% bounds violations but 126 velocity violations across 500 samples), ACT on ALOHA (transformer chunking, 14-DOF, 10 velocity violations), and Diffusion Policy on PushT (DDPM, 2-DOF, 100% OOB due to pixel-space actions at [44, 453] with default [-1, 1] bounds). The PushT case illustrates a second normalization mismatch scenario: the policy outputs pixel coordinates, not normalized actions. SafeContract detects this immediately from the violation pattern, regardless of architecture.

Normalization vs. monitoring. Action normalization (mapping model outputs to the target robot's action space) eliminates most bounds violations. SafeContract is complementary, not redundant: it provides velocity enforcement (a physical hardware constraint that normalization cannot address), violation traceability for debugging, and distribution shift detection. When normalization is correctly applied, SafeContract's bounds checks become a safety net; the velocity monitoring, logging, and fingerprinting remain the primary value.

Generalization to real robots. Per-joint bounds and velocity limits are physically grounded constraints that apply to every robot. The decorator wraps any Python predict() method, regardless of model internals. The primary value in real deployment is threefold: (1) catching integration bugs during initial deployment (the first indication of a normalization mismatch is currently physical

damage), (2) monitoring model updates for regression (violation fingerprint changes signal behavioral drift), and (3) providing an audit trail for safety certification. The monitoring signal is most informative during deployment transitions; in steady-state operation with correct normalization, violation rates approach zero and the safety net becomes a passive guarantee.

Limitations. (1) EXP-G and EXP-H use synthetic task distributions; real VLA violation patterns may differ. (2) No closed-loop evaluation: aggressive clipping could degrade task success. This is the most important next step. (3) The 100% OOB finding (EXP-A) primarily reflects normalization mismatch, not model instability. (4) Property II-C is an empirical observation, not a formal statistical guarantee.

Future work. PropertyGuard: property-based testing that fuzzes action sequences against contracts to discover edge cases before deployment. Violations as interpretability: heatmaps of violation patterns across models (SmolVLA vs. OpenVLA vs. pi0) to characterize model-level failure modes. Conformal prediction calibration for probabilistic safety certificates with finite-sample coverage guarantees. Closed-loop LIBERO evaluation with task success rates.

References

[1] Kim, M.J. et al., "OpenVLA: An Open-Source Vision-Language-Action Model," ECCV 2024, arXiv:2406.09246.

[2] HuggingFace, "SmolVLA: A Small Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025.

[3] Cadene, R. et al., "LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch," 2024, https://github.com/huggingface/lerobot.

[4] Physical Intelligence, "pi0: A General-Purpose Robot Foundation Model and OpenPI Deployment Framework," 2025.

[5] Meyer, B., "Applying 'Design by Contract'," Computer 25(10):40–51, 1992.

[6] Liu, B. et al., "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning," NeurIPS 2023.

[7] PKU-Alignment, "SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning," NeurIPS 2025, arXiv:2503.03480.

[8] VLSA/AEGIS, "Plug-and-Play Safety Constraint Layer for VLA Policies," arXiv:2512.11891, 2024.

[9] SafeDec, "Safe Decoding for Robot Manipulation via STL-Constrained Policy Generation," arXiv:2509.01728, 2025.

[10] Yang, Z. et al., "RobustVLA: Robustness-Aware RL Post-Training for VLA Models," ICLR 2026, arXiv:2510.00037.

[11] Yang, Z. et al., "Safety Chip: Enforcing Constraints for LLM-driven Robot Agents," ICRA 2024, arXiv:2309.09919.

[12] Wei, T. et al., "SafeDiffuser: Safe Planning with Diffusion Probabilistic Models," ICLR 2025, arXiv:2306.00148.

[13] Romero, L. et al., "CoDiG: Constraint-Aware Diffusion Guidance for Robotics," CoRL 2025, arXiv:2505.13131.

[14] Quackenbush, M. et al., "Safe Robot Foundation Models Using Inductive Biases," arXiv:2505.10219, 2025.

[15] Song, Y. et al., "LIBERO-Plus: In-depth Robustness Analysis of VLA Models," arXiv:2510.13626, 2025.

[16] PKU-Alignment, "VLA-Arena: Benchmarking VLA Models," arXiv:2512.22539, 2025.

[17] Bhardwaj, V.P., "Agent Behavioral Contracts," arXiv:2602.22302, 2026.

[18] Hu, H. et al., "Sentinel: Runtime Monitoring of Generative Policies via STAC," CoRL 2024, arXiv:2410.04640.

[19] SABER, "Black-Box Adversarial Attacks on VLAs via Instruction Perturbation," arXiv:2603.24935, 2026.